DSL Grammar Documentation for Simulation Sequencing

Frédéric Sabot, Sreejith P. Sajeev

1 Introduction

This document describes the Domain-Specific Language (DSL) used to sequence events in a running power system simulation. The reference grammar is `Orchestrator/DSL/PowerEventDSL.g4`; it is parsed with ANTLR and interpreted by `DSLEngine`, which runs inside the orchestrator process.

A script is a sequence of statements. The one named by `input_files.dsl_file` in the orchestrator configuration is loaded when the engine starts, and further statements can be submitted over the HTTP interface while the simulation runs (e.g. via Argus). Both go through the same parser and the same validation. Key features include:

- **Time-Based Events**: schedule an action at a time, or leave the time out to act at the next step.
- **Ramps**: apply a percentage change or a setpoint change progressively over an interval.
- **Condition-Based Events**: `when` and `when_rearming` watch live measurements and fire when a boolean expression over them becomes true.
- **Dependent Events**: `after "<tag>" is completed` links an action to earlier tagged work.
- **Targeting**: one element, a list, or a cluster named or defined by regular expression.
- **Network Actions**: `change network characteristics` on links and nodes, with `drop`, `delay` and `corruption` operators.
- **Functions**: reusable templates with parameters bound at invocation, and `validate` to expand a call and check it without applying anything.

Every statement ends with a semicolon (`;`). A `//` starts a comment that runs to end of line.

2 Script Structure

2.1 Statements

A script contains three kinds of top-level statement, in any order:

```
func <NAME>(<PARAM>, ...) { ... }          // function definition
def <TYPE> cluster "<ID>" <DEFINITION>;    // cluster definition
after 5 seconds, open switch "S1";         // event definition
```

Description: Statements are applied one at a time, in order. A statement that fails validation aborts the load and is reported, but the statements before it have already been applied and stay applied, the way a Python script leaves behind whatever ran before the line that raised. Re-submitting a corrected script therefore re-applies its leading statements a second time.

2.2 Time Expressions

Syntax:

```
after 30 seconds, ...      // an offset
at 30 seconds, ...         // an absolute simulation time

<TIME_EXPR> ::= <NUMBER> <TIME_UNIT> | $<PARAM> [<TIME_UNIT>]
<TIME_UNIT> ::= seconds | second | s | minutes | minute | min |
   hours | hour | h
```

Description: A time is introduced by one of two anchors:

- `after <TIME_EXPR>` is an offset. From when the statement is submitted, or, inside a function body, from the invocation (see section 7.3).
- `at <TIME_EXPR>` is an absolute simulation time. `at 300 seconds` is 300 seconds into the run whenever the statement is submitted. A time already past is rejected when the statement is loaded. `at` statements inside a function body are rejected to allow the function to be called at any time.

For a script loaded at the start of a run, `after` and `at` are equivalent. However, it is strongly recommended to prefer `after` as it works in a larger set of situations (allows the script to be launched after the start time, and works inside function bodies). `at` should be reserved for single commands launched from the terminal. A literal always carries a unit: `after 30` is a syntax error. A `$parameter` may carry one, and counts as seconds without.

3 Event Definitions

3.1 Point-in-Time Event

Syntax:

```
after <TIME_EXPR>, <EXECUTABLE> [tagged "<TAG>"];    // offset from submission
at    <TIME_EXPR>, <EXECUTABLE> [tagged "<TAG>"];    // absolute simulation time
```

Example:

```
after 2 seconds, open switch "01_1_breaker" tagged "load_1_isolated";
after 10 s, set load "02_1" input "P" to 250;
at 300 seconds, open line "01_2";
```

3.2 Immediate Event

Syntax:

```
<EXECUTABLE> [tagged "<TAG>"];
```

Description: With the anchor omitted the action is applied at the next tick, which is the same as writing `after 0 seconds`.

3.3 Interval Event (Ramp)

Syntax:

```
after [<TIME_EXPR>, <TIME_EXPR>] [with law linear], <RAMPABLE_ACTION> [tagged "<TAG>"];
at    [<TIME_EXPR>, <TIME_EXPR>] [with law linear], <RAMPABLE_ACTION> [tagged "<TAG>"];
```

Example:

```
after [2 seconds, 6 seconds] with law linear, decrease load "01_1" input "P" by 20%;
after [10 seconds, 14 seconds] with law linear, set load "02_1" input "P" from 100 to 300;
```

Description: Only two actions are rampable: the percentage adjustment and the `from ... to` setpoint change. `with law` is optional and `linear` is the only law implemented, so any other name is rejected. The ramp is advanced once per tick between its bounds; see section 4.6 for what each kind writes.

3.4 Condition-Based Events

Syntax:

```
when <CONDITION>, <EXECUTABLE> [tagged "<TAG>"];
when_rearming <CONDITION>, <EXECUTABLE> [tagged "<TAG>"];
```

Example:

```
when load "01_1" output "P" > 200, open switch "01_1_breaker" tagged
    "overload_trip";
when_rearming load "02_1" output "P" > 150, decrease load "02_1"
   input "P" by 5%;
```

Description: The condition is evaluated once per tick against the last value measured for each output it reads, see section 6. A `when` fires the first time the condition holds and is then completed. A `when_rearming` fires the same way but re-arms once the condition has gone false again, so it can fire repeatedly.

3.5 Dependent Event

Syntax:

```
after "<TAG>" is completed, <EXECUTABLE> [tagged "<TAG>"];
```

Example:

```
after "load_1_isolated" is completed, increase load "05_1" input "P"
    by 10%;
```

Description: Fires on the first tick at which every event carrying that tag has completed. A tag whose owner was deleted never completes.

3.6 Tags

Description:

- `tagged "<TAG>"` names the events a statement created, so later statements can wait on them.
- One statement can create several events, e.g. a function call. The tag then covers all of them and completes only once they all have.

4 Action Types

4.1 Switch Actions

Syntax:

```
open switch "<SWITCH_ID>";
close switch "<SWITCH_ID>";
```

Description: Writes the `state` attribute of the switch, `1` to open and `2` to close. The identifier is an IIDM id and must appear in the game master input mapping, otherwise the statement is rejected.

4.2 Line Actions

Syntax:

```
open line "<LINE_ID>";
close line "<LINE_ID>";
```

Description: As above, on a line: `state` is set to `1` to open and `2` to close. Note that such line action is meant to represent faults. It works by setting the line state to 'open' (2) in Dynawo which indirectly leads to the opening of the breakers on both sides of the line. The breakers cannot be manually reclosed without a "close line" action.

4.3 Transformer Actions

Syntax:

```
open transformer "<TRANSFORMER_ID>";
close transformer "<TRANSFORMER_ID>";
```

Description: As above, on a transformer. Works both on `2_windings_transformer` and `3_windings_transformer`.

4.4 Percentage Adjustment

Syntax:

```
increase <TARGET> input "<INPUT>" by <NUMBER>%;
decrease <TARGET> input "<INPUT>" by <NUMBER>%;
```

Example:

```
after 15 s, decrease load ["03_1", "04_1"] input "P" by 5%;
```

Description: Scales the element's input multiplier rather than writing a setpoint, so the change is relative to whatever the model is currently producing. `decrease ... by 5%` multiplies by 0.95.

4.5 Absolute Setpoint

Syntax:

```
set <TARGET> input "<INPUT>" to <SIGNED_NUMBER>;
```

Example:

```
after 6 seconds, set load cluster "spares" ["11_1", "12_1"] input "P
   " to 120;
```

Description: Writes the value directly.

4.6 Ramps

Syntax:

```
after [<T1>, <T2>] [with law linear], increase <TARGET> input "<INPUT>" by <NUMBER>%;
after [<T1>, <T2>] [with law linear], set <TARGET> input "<INPUT>" [from <SIGNED_NUMBER>] to <SIGNED_NUMBER>;
```

Example:

```
after [5 seconds, 9 seconds], set load "02_1" input "P" from 200 to 300;
after [5 seconds, 9 seconds], set load "02_1" input "P" to 300;
```

Description:

- The percentage form is a *multiplier* ramp.
- The `to` form is a *value* ramp. With `from` it starts from the number written; without, it starts from the element's current measured value. Omitting it needs the element to publish a measurement for that input, which is checked when the statement is loaded, so a ramp that could never start is refused rather than failing silently later.
- Two ramps may not overlap in time on the same element and input. The second one is rejected when it is loaded.
- A zero-length interval applies the whole change in one tick.

4.7 Network Actions

These modify the communication network carrying measurements and controls.

General Syntax

```
change network characteristics
   link from "<SRC>" to "<DST>" [, link from "<SRC>" to "<DST>"]*
   [, node "<NODE>"]*
   [, <OPERATOR>]* ;

change network characteristics
   node "<NODE>" [, node "<NODE>"]*
   [, <OPERATOR>]* ;
```

At least one link or node clause is required, and clauses are separated by commas. Several operators can be composed in one statement.

Operators

- `drop`: lose information passing through the link or node.

```
drop fixed <true|false>  // Drop everything or nothing
drop random <PROBABILITY>  // Drop with a probability (between 0 and 1)
```

- `delay`: latency injection

```
delay fixed <NUMBER> <TIME_UNIT>
delay gauss mean <NUMBER> <TIME_UNIT> stddev <NUMBER> <
   TIME_UNIT>
```

- `corruption`: data corruption

```
corruption none [likelihood <PROBABILITY>]
corruption random_noise_measurement [likelihood <PROBABILITY>]
   range <LOW> to <HIGH>
corruption random_noise_control [likelihood <PROBABILITY>]
   range <LOW> to <HIGH>
corruption zero_out_measurement [likelihood <PROBABILITY>] [
   value <NUMBER>]
corruption shift_measurement [likelihood <PROBABILITY>] by <
   NUMBER>
corruption invert_control [likelihood <PROBABILITY>] [by <
   NUMBER>]
```

Example

```
after 1 seconds, change network characteristics
  link from "CC" to "substation_1",
  drop fixed true
  tagged "CC_to_substation_1";

after "CC_to_substation_1" is completed, change network
   characteristics
  link from "CC" to "substation_2", link from "CC" to "substation_3"
   , node "substation_2",
  delay fixed 0.15 seconds;

after 8 seconds, change network characteristics
  link from "CC" to "substation_2", node "substation_2",
  corruption shift_measurement likelihood 0.2 by 0.02;
```

5 Target Specifications

5.1 Element Types

Every target names an element type first. The accepted types follow pypowsybl nomenclature:

```
2_windings_transformer    phase_tap_changer
3_windings_transformer    ratio_tap_changer
battery                   shunt_compensator
```

```
dangling_line              static_var_compensator
generator                  substation
hvdc_line                  switch
lcc_converter_station      tie_line
line                       voltage_level
load                       vsc_converter_station
```

These are reserved keywords, so none of them can be used as a function or parameter name. The same applies to every other quoted literal in the grammar, `second` and `line` included.

5.2 Single Element and Lists

Syntax: `<TYPE> "<ID>"` or `<TYPE> ["<ID>", "<ID>", ...]`

Example:

```
after 15 s, decrease load ["03_1", "04_1"] input "P" by 5%;
```

Description: A list is one action on several elements, exactly like the anonymous cluster below, so the statement is a single event however many elements it names. Cancelling it cancels the whole statement.

5.3 Clusters

Syntax:

```
def <TYPE> cluster "<NAME>" <DEFINITION>;       // named, reusable
<TYPE> cluster "<NAME>"                         // use a named
   cluster
<TYPE> cluster "<NAME>" <DEFINITION>            // define and use in
   one statement
<TYPE> cluster <DEFINITION>                     // anonymous

<DEFINITION> ::= ["<ID>", "<ID>", ...] | regex "<PATTERN>" on <TYPE>
```

Example:

```
def load cluster "feeders" ["01_1", "02_1", "03_1"];

after 3 seconds, decrease load cluster "feeders" input "P" by 10%;
after 6 seconds, set load cluster "spares" ["11_1", "12_1"] input "P
   " to 120;
after 9 seconds, increase load cluster regex "^4[0-9]_1$" on load
   input "P" by 5%;
```

Description: A regular expression is matched against the ids of the network's elements of that type using search semantics, so anchor it with `^` and `$` to require a full match.

5.4 Dynamic Models

Syntax: "<MODEL_LIBRARY>" "<MODEL>"

Example:

```
after 2 seconds, set "DYNModelRST" "RST_VL_A_225" input "U" to 400;
after 5 seconds, decrease "DYNModelRST" "RST_VL_A_225" input "deltaU
   " by 10%;
```

Description: Some Dynawo models are purely dynamic and are not attached to an IIDM element. Those are named by their model type and model id instead (e.g. "DYNModelRST" "RST_VL_A_225" instead of line "L1").

Apart from this, they work the same, e.g. in cluster, regexes, function calls, etc.

```
after 12 seconds, set "DYNModelRST" ["RST_VL_A_225", "RST_VL_B_225"]
    input "U" to 402;
def "DYNModelRST" cluster "regulators" regex "^RST_" on "DYNModelRST
   ";
after 18 seconds, increase "DYNModelRST" cluster "regulators" input
   "U" by 1%;
when "DYNModelOmegaRef" "OmegaRef" output "OmegaRef" < 49.8,
     set "DYNModelRST" "RST_VL_A_225" input "deltaU" to 2;
```

6 Dynamic Conditions

6.1 Reading a Measurement

Syntax:

```
<TYPE> "<ELEMENT_ID>" output "<ATTRIBUTE>"
"<MODEL_LIBRARY>" "<MODEL>" output "<ATTRIBUTE>"    // purely dynamic
    model
```

6.2 Expressions

Syntax:

```
<CONDITION>  ::= <TERM> (or <TERM>)*
<TERM>       ::= <FACTOR> (and <FACTOR>)*
<FACTOR>     ::= <COMPARISON> | true | false | ( <CONDITION> )
<COMPARISON> ::= <ARITHMETIC> (< | > | <= | >= | == | !=) <
   ARITHMETIC>
<ARITHMETIC> ::= <NUMBER> | $<PARAM> | <MEASUREMENT> | arithmetic
   with + - * / and parentheses
```

Example:

```
when load "01_1" output "P" > 200, open switch "01_1_breaker";
when (load "01_1" output "P" + load "02_1" output "P") > 400 and
   voltage_level "01" output "U" < 0.95,
     open line "01_2";
```

7 Functions

7.1 Function Definition

Syntax:

```
func <NAME>(<PARAM>, <PARAM>, ...) {
   <CLUSTER_DEFINITION_OR_EVENT>
   ...
}
```

Example:

```
func isolateFeeder(breaker, feeder) {
  after 0 seconds, open switch $breaker tagged "breaker_opened";
  after 3 seconds, decrease load $feeder input "P" by 15%;
  after "breaker_opened" is completed, set load $feeder input "P" to
    80;
}
```

Description:

- A function body can contain any list of statements, including cluster definitions, `when` and `after` statements, and calls to other functions.
- Recursive function calls are not allowed.
- A parameter is referenced as `$name`. It holds a plain value, a string, a number or a boolean.
- Definitions live for the rest of the session.

7.2 Function Invocation

Syntax:

```
<NAME>(<ARG>, ...);
<NAME>(<PARAM>: <ARG>, ...);
validate(<NAME>(<ARG>, ...));
```

Example:

```
after 5 seconds, isolateFeeder("01_1_breaker", "01_1");
after 20 seconds, isolateFeeder(breaker: "02_1_breaker", feeder: "02_1");
```

Description:

- Arguments may be positional or named, in any order. Named ones bind by parameter name.

7.3 Relative Timing

Example:

```
func stagedIsolation(breaker, feeder) {
  after 6 seconds, open switch $breaker tagged "breaker_opened";
  after 8 seconds, decrease load $feeder input "P" by 10%;
  after "breaker_opened" is completed, set load $feeder input "P" to 60;
}

after 3 seconds, stagedIsolation("01_1_breaker", "01_1");
```

Description: Times inside a body are relative to the invocation. Above, the switch opens at 9 seconds and the adjustment lands at 11. Nesting composes: a call made `after 5 seconds` inside a body invoked at 3 seconds has its own body counted from 8.

A function body accepts `after` only. `at` is rejected when the call is loaded, because a function is a template invoked at an arbitrary time and an absolute time in its body would pin it to one moment and make it unusable anywhere else.

7.4 Verification with validate

Syntax:

```
validate(<FUNCTION_CALL>);
```

Description: Expands the call and validates every event it would create in the current scope. Nothing is scheduled nor applied. An error is reported exactly as it would have been for the real call, which makes it a dry run for a sequence that is expensive to undo.

8 Errors and Validation

8.1 Syntax Errors

Missing semicolons, unknown keywords, or a reserved keyword used as an identifier. Reported by ANTLR with the line and the offending token.

8.2 Validation Errors

Raised while a statement is being loaded: an element or attribute that is not in the game master mapping, an unknown cluster, function or tag, a ramp overlapping an existing one on the same element and input, a ramp law other than `linear`, a call argument that would be ignored, or a parameter used where its value has the wrong type. The message carries the line number and the source text of the statement that failed.

8.3 Runtime Behaviour

A regular expression that matches nothing yields an empty cluster and a statement that does nothing. A condition over an output that has not been measured yet is false rather than an error.

9 ANTLR Implementation

The DSL is powered by ANTLR:

- **Grammar File**: `Orchestrator/DSL/PowerEventDSL.g4`
- **Generated Classes**: Lexer (`PowerEventDSLLexer`), Parser (`PowerEventDSLParser`), Visitor (`PowerEventDSLVisitor`), generated into `Orchestrator/DSL/generated/` at build time.
- **Interpretation**: `DSLProgramVisitor` walks the parse tree and produces `DSLEvent` objects, which `DSLEngine` schedules and evaluates.

10 Example Scripts

Each of these is a case in `Orchestrator/DSL/tests/cases`, run by the regression suite against the Nordic example network. Other examples are included in this test suite.

10.1 Point Actions

```
after 2 seconds, open switch "01_1_breaker" tagged "load_1_isolated"
    ;
after 5 seconds, close switch "01_1_breaker";
after 10 s, set load "02_1" input "P" to 250;
after 15 s, decrease load ["03_1", "04_1"] input "P" by 5%;
after "load_1_isolated" is completed, increase load "05_1" input "P"
    by 10%;
```

10.2 Clusters

```
def load cluster "feeders" ["01_1", "02_1", "03_1"];

after 3 seconds, decrease load cluster "feeders" input "P" by 10%;
after 6 seconds, set load cluster "spares" ["11_1", "12_1"] input "P" to 120;
after 9 seconds, increase load cluster regex "^4[0-9]_1$" on load input "P" by 5%;
```

10.3 Ramps

```
after [2 seconds, 6 seconds] with law linear, decrease load "01_1" input "P" by 20%;
after [10 seconds, 14 seconds] with law linear, set load "02_1" input "P" from 100 to 300;
```

10.4 Conditional Triggers

```
when load "01_1" output "P" > 200, open switch "01_1_breaker" tagged "overload_trip";
when_rearming load "02_1" output "P" > 150, decrease load "02_1" input "P" by 5%;
```

10.5 Functions

```
func isolateFeeder(breaker, feeder) {
  after 0 seconds, open switch $breaker tagged "breaker_opened";
  after 3 seconds, decrease load $feeder input "P" by 15%;
  after "breaker_opened" is completed, set load $feeder input "P" to 80;
}

func trimGeneration(feeder, amount) {
  after [0 seconds, 4 seconds] with law linear, decrease load $feeder input "P" by $amount%;
}

after 5 seconds, isolateFeeder("01_1_breaker", "01_1");
after 20 seconds, isolateFeeder(breaker: "02_1_breaker", feeder: "02_1");
after 30 seconds, trimGeneration("03_1", 12);
```

10.6 Parameters of Every Kind

```
func openOne(breaker) {
  after 0 seconds, open switch $breaker;
}

func openPair(left, right) {
  openOne($left);
  openOne($right);
}

func trim(group, attribute, amount) {
  after 0 seconds, decrease load cluster $group input $attribute by
   $amount%;
}

def load cluster "feeders" ["01_1", "02_1"];

after 2 seconds, openOne("01_1_breaker");
after 5 seconds, openPair("02_1_breaker", "03_1_breaker");
after 8 seconds, trim("feeders", "P", 10);
```